

ANCESTRAL CHARACTER ESTIMATION

Alex Beams

Department of Mathematics
Simon Fraser University

Phylogeography Workshop

Day 1

- 1 LIKELIHOOD FOR MARKOV MODELS ON TREES
- 2 ANCESTRAL STATE RECONSTRUCTION
- 3 FITTING MODELS IN R

TABLE OF CONTENTS

- 1 LIKELIHOOD FOR MARKOV MODELS ON TREES
- 2 ANCESTRAL STATE RECONSTRUCTION
- 3 FITTING MODELS IN R

MARKOV MODELS FOR DISCRETE STATES ON TREES

Markov chains are useful for reconstructing phylogenies from genomic data, but let's start with a simpler problem first (and actually, one which arises frequently in phylogeographic analysis/ancestral state reconstruction):

Suppose we already have a phylogenetic tree and information about discrete traits at the tips (the data we collect). How do we quantify the probabilities of different scenarios of trait evolution, starting at the most recent common ancestor, down to our sampled/observed individuals?

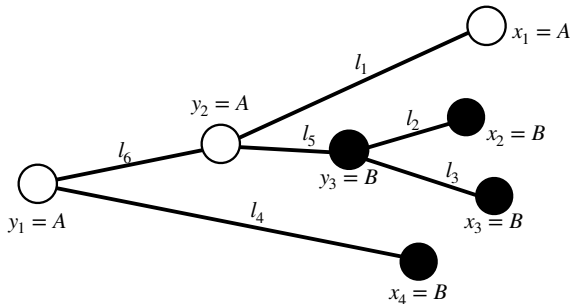
THE APPROACH TO RECONSTRUCTING ANCESTRAL STATES

Ancestral state reconstruction relies on basic ideas from statistics.

- 1 Propose a probabilistic model for trait evolution on the tree
 - 1A The model needs to integrate/sum over over all possible (unobserved) evolutionary scenarios down to the present day
- 2 Fit the model to the data by estimating parameters governing rates of trait change over time
 - 2A Use Maximum Likelihood to choose the best parameter estimates
- 3 Identify the (unobserved) evolutionary scenario that maximizes the probability of the data we observe under the model

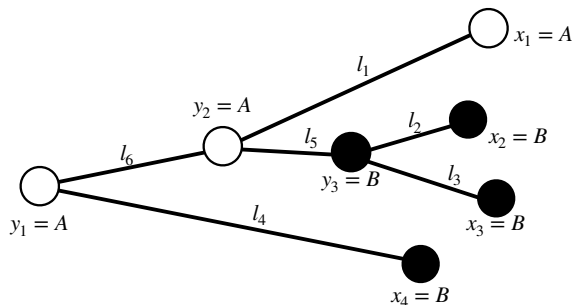
MARKOV MODELS FOR DISCRETE STATES ON TREES

Example: suppose we are given a phylogenetic tree, $\mathcal{T}$, we have information about tip locations, $x_i \in \{A, B\}$, and we superimpose a continuous-time Markov chain with the two states (A and B) on the tree



If we knew the locations of internal nodes, y_i , we could calculate transition probabilities along each branch and combine these to calculate the probability of observing the tip data

MARKOV MODELS FOR DISCRETE STATES ON TREES



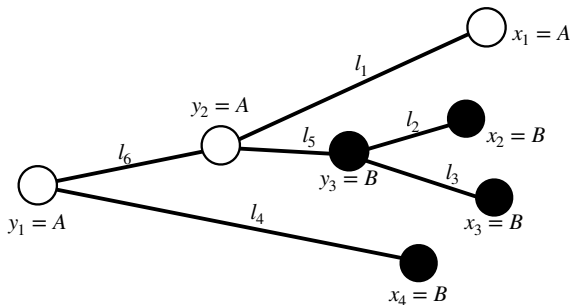
Example: transition from $y_1 = A$ to $x_4 = B$:

$$p(x_4|y_1) =$$

$$p(X_4 = B|Y_1 = A, l_4, Q) = e^{Ql_4} P_0,$$

with $P_0 = (1, 0)^T$ because y_1 is assumed to reside in state A

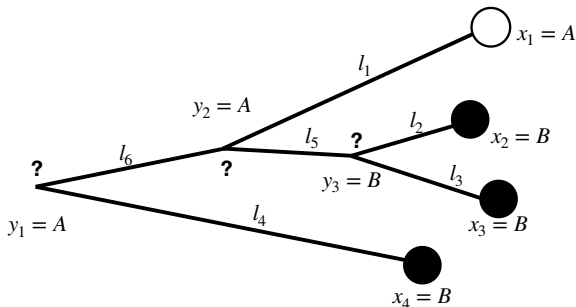
MARKOV MODELS FOR DISCRETE STATES ON TREES



Repeating this along each branch, we could calculate the probability of observing the joint configuration of tip states, x_i , and node states, y_i :

$$P(\vec{x}, \vec{y} | \mathcal{T}) = \pi(y_1) p(x_4 | y_1) p(y_2 | y_1) p(x_1 | y_2) p(y_3 | y_2) p(x_2 | y_3) p(x_3 | y_3)$$

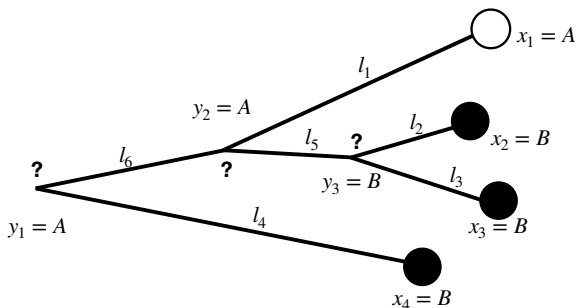
CALCULATING LIKELIHOOD



However, there is a problem: we don't actually observe/know the the values of internal nodes, y_i , so we have to sum over all possible configurations of the internal nodes:

$$P(\vec{x}|\mathcal{T}) = \sum_{\vec{y}} \pi(y_1)p(x_4|y_1)p(y_2|y_1)p(x_1|y_2)p(y_3|y_2)p(x_2|y_3)p(x_4|y_3)$$

CAN FIT TO DATA USING MAXIMUM LIKELIHOOD



The transition probabilities $p(y_j|y_i)$, $p(x_k|y_j)$ depend on parameters, θ , of a continuous-time Markov chain that we specify; if we interpret $P(\vec{x}|\mathcal{T}, \theta)$ as a function of θ , then we can fit the Markov model to the data $(\mathcal{T}, \vec{x})$ using Maximum Likelihood

THE TREE STRUCTURE ALLOWS US TO “PRUNE”

How do we calculate $p(\mathbf{x}|\mathcal{T})$? The likelihood can be calculated using the *pruning* (or, dynamic programming) method:

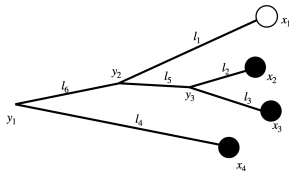
$$\begin{aligned} \sum_{\vec{y}} \pi(y_1) p(x_4|y_1) p(y_2|y_1) p(x_1|y_2) p(y_3|y_2) p(x_2|y_3) p(x_4|y_3) = \\ \sum_{y_1} \pi(y_1) p(x_4|y_1) \times \\ \sum_{y_2} p(y_2|y_1) p(x_1|y_2) \times \\ \sum_{y_3} p(y_3|y_2) p(x_2|y_3) p(x_3|y_3) \end{aligned}$$

The calculation is carried from right to left, starting with the marginalization in y_3 , then in y_2 , then in y_1 . This makes the calculation feasible for large trees

PRUNING

The form of the pruned likelihood indicates that we calculate from the tips toward the root. First, we calculate the likelihood of the tree descending from node y_3 , conditional on the state being z ,

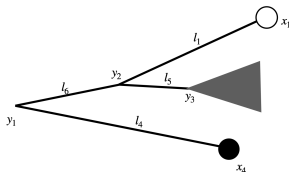
$$L_{y_3}(z) = p(x_2|y_3 = z, l_2)p(x_3|y_3 = z, l_3) :$$



PRUNING

The notation $L_{y_k}(z)$ refers to the likelihood of the tree descending from node y_k , given that y_k has state z .

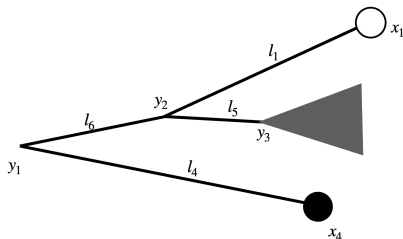
So we have as many terms $L_{y_3}(z)$ as there are possible values, z , for the node y_3



PRUNING

Once we have $L_{y_3}(z)$, we can calculate $L_{y_2}(z)$:

$$L_{y_2}(z) = p(x_1|y_2 = z, l_1) \left(\sum_w p(y_3 = w|y_2 = z, l_5) L_{y_3}(w) \right)$$

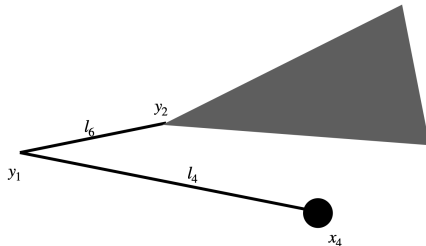


PRUNING

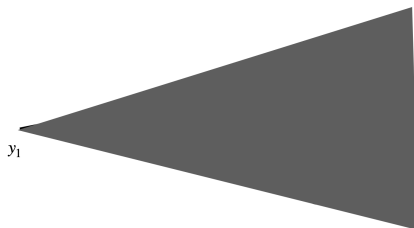
Finally, we get to $L_{y_1}(z)$:

$$L_{y_1}(z) = p(x_4|y_1 = z, l_4) \left(\sum_u p(y_2 = u|y_1 = z, l_6) L_{y_2}(u) \right),$$

and we have contributions for each of the possible values of $y_1 = z$.



PRUNING



To obtain the tree likelihood, we combine the likelihoods of the tree conditional on y_1 being in state z . This is accomplished by setting

$$L_{y_1} = \sum_{y_1} \pi(z) L_{y_1}(z).$$

for some distribution π describing the probability that the root is in state z .

PRUNING

$$L_{y_1} = \sum_z \pi(z) L_{y_1}(z).$$

In ace, the distribution π is chosen to be the stationary distribution of the Markov chain (which ensures a detailed balance condition holds throughout the tree whenever the Markov model is reversible)

The $\pi(y_1)$ is necessary for the tree likelihood to be interpreted as a marginalization of a joint distribution; there is no Bayesian interpretation here (or at least, there does not need to be)

PRUNING

$$L_{y_1} = \sum_{y_1} \pi(y_1) L_{y_1}(z).$$

In the phytools functions (e.g. `make.simmap`), the distribution π can be the stationary distribution of the Markov chain like `ace`, but can also be the “FitzJohn root prior”¹, or a user-supplied distribution

For reversible Markov chains (irreducible, aperiodic, and positive-recurrent), selecting π to be the stationary distribution ensures detailed balance throughout the phylogeny

¹Richard G FitzJohn, Wayne P Maddison, and Sarah P Otto. “Estimating trait-dependent speciation and extinction rates from incompletely resolved phylogenies”. In: *Systematic biology* 58.6 (2009), pp. 595–611.

MARKOV MODELS FOR BINARY TRAIT EVOLUTION

Fun side note: in the original presentation of these models, Pagel was interested to study correlated evolution of two or more traits.

If the traits evolve in a correlated manner, state changes between the four combinations of states can be modeled with a 4×4 Markov generator (up to 12 parameters):

$$Q = \begin{pmatrix} -\sum_{j \neq 1} \lambda_{1j} & \lambda_{12} & \lambda_{13} & \lambda_{14} \\ \lambda_{21} & -\sum_{j \neq 2} \lambda_{2j} & \lambda_{23} & \lambda_{24} \\ \lambda_{31} & \lambda_{32} & -\sum_{j \neq 3} \lambda_{3j} & \lambda_{34} \\ \lambda_{41} & \lambda_{42} & \lambda_{43} & -\sum_{j \neq 4} \lambda_{4j} \end{pmatrix}$$

MARKOV MODELS FOR BINARY TRAIT EVOLUTION

If, on the other hand, the traits evolve independently of each other (“null hypothesis”), state changes between the four combinations of states are modeled with two independent Markov chains, each with a 2×2 Markov generator (4 parameters):

$$Q^{(1)} = \begin{pmatrix} -\lambda_{12}^{(1)} & \lambda_{21}^{(1)} \\ \lambda_{12}^{(1)} & -\lambda_{21}^{(1)} \end{pmatrix} \quad Q^{(2)} = \begin{pmatrix} -\lambda_{12}^{(2)} & \lambda_{21}^{(2)} \\ \lambda_{12}^{(2)} & -\lambda_{21}^{(2)} \end{pmatrix}$$

If the more complicated model fits significantly better (using, e.g., a likelihood ratio test), that suggests trait evolution is indeed correlated.

MARKOV MODELS FOR BINARY TRAIT EVOLUTION

For our purposes, we will primarily focus on evolution of individual traits. The overall goal is to understand how we can use the model to estimate the most likely configuration of unobserved ancestral states, given the data and the model.

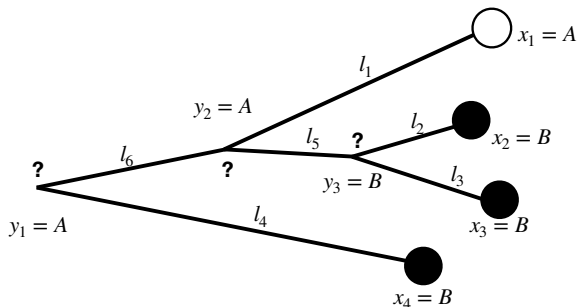
TABLE OF CONTENTS

- 1 LIKELIHOOD FOR MARKOV MODELS ON TREES
- 2 ANCESTRAL STATE RECONSTRUCTION
- 3 FITTING MODELS IN R

ANCESTRAL STATE RECONSTRUCTION

To reconstruct ancestral states, we use the likelihood function we just obtained. Let's examine the form of the likelihood a little more carefully:

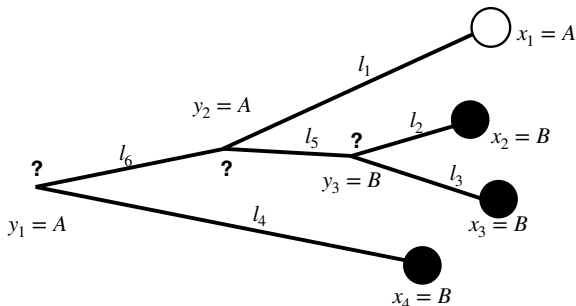
MARKOV MODELS FOR DISCRETE STATES ON TREES



The **likelihood** of the model is the probability of observing the tip states, $\mathbf{x} = (x_1, x_2, \dots, x_n)^T$ given the branch lengths l_i and parameters of the Markov model in $\mathbf{Q}$:

$$P(\vec{\mathbf{x}}|\mathcal{T}) = \sum_{\mathbf{y}} \pi(y_1) p(x_4|y_1) p(y_2|y_1) p(x_1|y_2) p(y_3|y_2) p(x_2|y_3) p(x_4|y_3)$$

CALCULATING LIKELIHOOD



Because don't know the the states of internal nodes, y_i , we have to sum over all possible configurations, $\mathbf{y}$ (and we saw that we could use the “pruning algorithm” to do this efficiently):

$$P(\vec{x}|\mathcal{T}) = \sum_{\mathbf{y}} \pi(y_1)p(x_4|y_1)p(y_2|y_1)p(x_1|y_2)p(y_3|y_2)p(x_2|y_3)p(x_4|y_3)$$

THE LIKELIHOOD IS A MARGINALIZATION OVER LATENT VARIABLES

Even though there are a lot of conditionals ($p(y_k|y_j)$) in the expression of the likelihood,

$$P(\vec{x}|\mathcal{T}) = \sum_{\mathbf{y}} \pi(y_1)p(x_4|y_1)p(y_2|y_1)p(x_1|y_2)p(y_3|y_2)p(x_2|y_3)p(x_4|y_3),$$

don't be confused – this is still a **marginalization of a joint distribution**. It has the form

$$p(\mathbf{x}|\mathcal{T}) = \sum_{\mathbf{y}} p(\mathbf{x}, \mathbf{y}|\mathcal{T}).$$

The likelihood is **not** of the form $p(\mathbf{x}|\mathcal{T}, \mathbf{y})$

In the likelihood, $\mathbf{x}$ is the configuration of tip states we observe, and $\mathbf{y}$ is the configuration of internal node states we do not observe.

ANCESTRAL STATES AS LATENT VARIABLES

The basic form of this likelihood (a marginalization over latent/unobserved variables, $\mathbf{y}$),

$$p(\mathbf{x}|\mathcal{T}) = \sum_{\mathbf{y}} p(\mathbf{x}, \mathbf{y}|\mathcal{T}),$$

immediately suggests how we should determine ancestral states, $\mathbf{y}$.

Our best estimate for the ancestral state configuration is the particular $\mathbf{y}^*$ that makes the largest contribution to $p(\mathbf{x}|\mathcal{T})$.

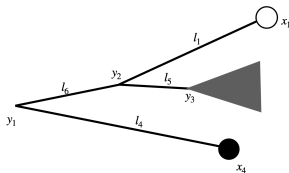
This is the **joint reconstruction** of ancestral states

This is a hard problem. For m traits and n tips in the tree, there are m^{n-1} different configurations of internal nodes. A generalization of the Viterbi algorithm makes it tractable²

²Tal Pupko et al. "A fast algorithm for joint reconstruction of ancestral amino acid sequences". In: *Molecular biology and evolution* 17.6 (2000), pp. 890–896.

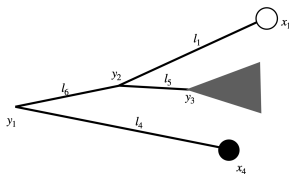
AN ALGORITHM TO FIND THE JOINT RECONSTRUCTION

When we calculated the Likelihood, we moved from the tips up to the root, like so:



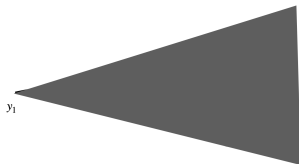
For this example tree, we would have conditional likelihoods $L_{y_3}(z)$, $L_{y_2}(z)$, and $L_{y_1}(z)$ for each value z (values of the possible ancestral states).

AN ALGORITHM TO FIND THE JOINT RECONSTRUCTION



The key to finding the joint reconstruction is this: in addition to calculating these conditional likelihoods to combine them later, we store them separately for a second pass from the root back to the tips

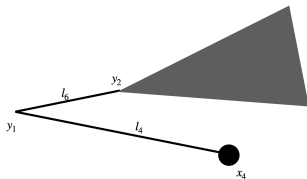
AN ALGORITHM TO FIND THE JOINT RECONSTRUCTION



When we get to the root in the pruning algorithm, we will have conditional likelihoods $L_{y_1}(z_1)$, $L_{y_1}(z_2)$, ... $L_{y_1}(z_m)$ for each of the m possible states at the root.

Start by setting $\hat{y}_1 = \operatorname{argmax}_z \pi(z) L_{y_1}(z)$ as the most likely root state. We proceed back down to the tips:

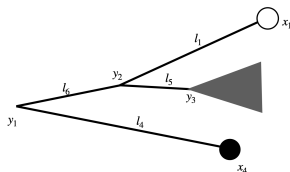
AN ALGORITHM TO FIND THE JOINT RECONSTRUCTION



Next, we need to select the state at y_2 . To do so, we use the state we chose at the root, $\hat{y}_1$, and calculate $p_{\hat{y}_1, z}(l_6)L_{y_2}(z)$ for each z (a transition probability multiplied by the likelihood of the subtree descending from y_2).

We set $\hat{y}_2 = \operatorname{argmax}_z p_{\hat{y}_1, z}(l_6)L_{y_2}(z)$

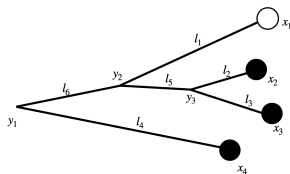
AN ALGORITHM TO FIND THE JOINT RECONSTRUCTION



Next, we need to select the state at y_3 . We use the state we chose at the parent node, $\hat{y}_2$. We calculate $p_{\hat{y}_2, z}(l_5)L_{y_3}(z)$ for each z .

We set $\hat{y}_3 = \operatorname{argmax}_z p_{\hat{y}_2, z}(l_5)L_{y_3}(z)$

AN ALGORITHM TO FIND THE JOINT RECONSTRUCTION



Now, we have $\mathbf{y}^* = (\hat{y}_1, \hat{y}_2, \hat{y}_3)$ as our joint reconstruction. We had to calculate m quantities at each internal node, which means the calculation is linear in m (the number of states in the Markov chain).

If you have worked with Hidden Markov models, you might recognize this as a generalization of the Viterbi algorithm.

MARGINAL RECONSTRUCTION OF ANCESTRAL STATES

So much for determining the most likely configuration of internal nodes, $\mathbf{y}^*$ (the **joint reconstruction**).

Alternatively, we may want to know the most likely state of an individual node, y_k , independent of the other nodes. This **marginal ancestral state reconstruction**³ proceeds by calculating, for each internal node y_k , the expression

$$p(\mathbf{x}, y_k | \mathcal{T}) = \sum_{y_1, y_2, \dots, y_{k-1}, y_{k+1}, \dots, y_n} p(\mathbf{x}, y_1, y_2, \dots, y_{k-1}, y_k, y_{k+1}, \dots, y_n | \mathcal{T}).$$

The value of y_k that corresponds to the largest $p(\mathbf{x}, y_k | \mathcal{T})$ is the **marginal ancestral character estimate** for node y_k .

³Dolph Schluter et al. "Likelihood of ancestor states in adaptive radiation". In: *Evolution* 51.6 (1997), pp. 1699–1711, Joseph Felsenstein. "Inferring phylogenies". In: *Inferring phylogenies*. 2004, pp. 664–664.

MARGINAL ANCESTRAL STATE ESTIMATION

$$L_{y_1} = \sum_{y_1} \pi(y_1) L_{y_1}(z).$$

The individual $L_{y_1}(z)$ terms in the likelihood suggest a way to carry out **marginal ancestral character estimation** for the root.

Just to reiterate: $L_{y_1}(z)$ is the likelihood of the tree conditional on the root taking on state z

The value of z that makes the largest contribution to L_{y_1} is the most likely state. We select it as our **marginal** estimate for the root state.

MARGINAL ANCESTRAL STATE ESTIMATION

So, we can find a marginal ancestral character estimate for the root of the tree, but what about all of the other nodes?

Whenever the Markov model of trait change is reversible (it usually is), the tree can actually be re-rooted at any node. So we just re-root, and then carry out the pruning algorithm each time⁴.

In this way, it is possible to calculate

$$p(\mathbf{x}, y_k | \mathcal{T}) = \sum_{y_1, y_2, \dots, y_{k-1}, y_{k+1}, \dots, y_n} p(\mathbf{x}, y_1, y_2, \dots, y_{k-1}, y_k, y_{k+1}, \dots, y_n | \mathcal{T}),$$

for each y_k . The value of y_k that gives the largest value for this expression is the marginal estimate.

⁴Joseph Felsenstein. "Inferring phylogenies". In: *Inferring phylogenies*. 2004, pp. 664–664.

JOINT VS. MARGINAL ANCESTRAL STATE ESTIMATION

Marginal character estimation is certainly tractable, but does not always produce the same estimates as the joint reconstruction⁵; they are designed to address different types of questions

In the ace function in R, there is an optional command “marginal” that defaults to FALSE, so that ace carries out a joint reconstruction by default

⁵Tal Pupko et al. “A fast algorithm for joint reconstruction of ancestral amino acid sequences”. In: *Molecular biology and evolution* 17.6 (2000), pp. 890–896.

TABLE OF CONTENTS

- 1 LIKELIHOOD FOR MARKOV MODELS ON TREES
- 2 ANCESTRAL STATE RECONSTRUCTION
- 3 FITTING MODELS IN R

WORKING IN R

We will use the R package ape to simulate and fit these models to example datasets (and real ones) Make sure you have the following

R packages installed:

- ape
- phytools

SOME IMPORTANT FUNCTIONALITY

The accompanying R demos will walk you through this in more detail, but it is worth mentioning here that the various functions we will use allow you to

- estimate a Markov generator, Q , with whatever constraints on rates you care to include (“equal rates” and “all rates different” being two popular defaults)
- carry out ancestral state reconstruction using joint or marginal approaches (with some caveats – see the demos!)

WORKING IN R

In the practical, you will

- Fit Markov models to trees with tip states
- Simulate fitted Markov models
- Explore some example datasets